

Generative AI Assessment

Bechir Zammouri

January 3, 2026

1 Graph Schema Overview

The healthcare knowledge graph is built using Neo4j, a property graph database that models entities as nodes and their connections as relationships. Figure 1 illustrates the complete graph schema, while Figure 2 shows a sample instance of the graph structure.

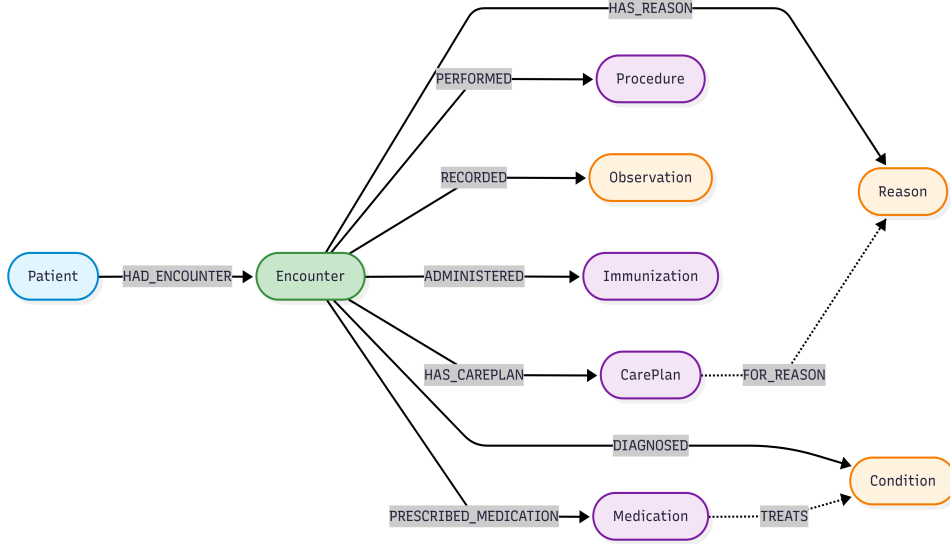


Figure 1: Graph Schema Diagram

1.1 Data Source

The healthcare data used in this project was sourced from Hugging Face’s **ajitnelson/synthetic-mass-health-data** dataset. Given the large scale of the original dataset, a representative sample was extracted to construct the knowledge graph, ensuring comprehensive coverage of various healthcare entities while maintaining computational efficiency.

1.2 Node Types

The graph contains nine primary node types representing healthcare entities:

- **Patient:** Contains demographic and identification data
- **Encounter:** Healthcare visits and interactions (central hub)
- **Reason:** Chief complaints and encounter reasons
- **Medication:** Pharmaceutical prescriptions and drugs

-
- **Condition:** Medical diagnoses and health conditions
 - **Procedure:** Medical procedures performed
 - **Observation:** Clinical measurements and vital signs
 - **Immunization:** Vaccination records
 - **CarePlan:** Treatment plans and care strategies

1.3 Graph Architecture

This graph follows an **encounter-centric model** where patients connect only to encounters, and all clinical events are accessed through encounters. This eliminates redundancy and provides clear temporal context.

1.4 Relationships

Relationships connect nodes to form a meaningful healthcare network:

- **HAD_ENCOUNTER:** Patient → Encounter (only direct patient relationship)
- **HAS_REASON:** Encounter → Reason
- **PRESCRIBED_MEDICATION:** Encounter → Medication
- **DIAGNOSED:** Encounter → Condition
- **PERFORMED:** Encounter → Procedure
- **RECORDED:** Encounter → Observation
- **ADMINISTERED:** Encounter → Immunization
- **HAS_CAREPLAN:** Encounter → CarePlan
- **FOR_REASON:** CarePlan → Reason
- **TREATS:** Medication → Condition

Each relationship carries temporal and contextual properties enabling rich querying capabilities. Detailed Cypher query implementations and examples are available in the GitHub repository's README file.

2 Agentic AI System

The healthcare knowledge graph is queried through an intelligent agent system that employs the **ReAct (Reasoning + Acting)** paradigm. This approach enables the agent to iteratively reason about user queries and take actions using specialized tools.

2.1 ReAct Architecture

The ReAct framework combines two key capabilities:

- **Reasoning:** The LLM analyzes the user's query, interprets previous tool results, and determines the next action
- **Acting:** The agent executes selected tools to retrieve information from the knowledge graph

This creates a dynamic loop: *Query* → *Reason* → *Act* → *Observe* → *Reason* → *Act* until sufficient information is gathered to answer the user's question.

2.2 LangGraph Workflow

The agent is implemented using LangGraph, a state machine framework for building agentic workflows. The workflow consists of three key components:

1. Agent State: A TypedDict containing conversation history (messages), query metadata, tool calls, and intermediate results. The state flows through the graph and accumulates context at each step.

2. Agent Node: Invokes the LLM with the current state and available tools. The LLM decides whether to call a tool or provide a final answer based on the conversation history.

3. Tools Node: Executes the tool(s) selected by the agent and appends results to the message history.

The workflow uses conditional routing: after each agent invocation, a decision function checks if tool calls are present. If yes, execution routes to the tools node; otherwise, the workflow terminates and returns the final answer.

2.3 Custom Tools

Two specialized tools provide comprehensive access to the healthcare knowledge graph:

2.3.1 Neo4j Query Tool

Executes Cypher queries against the graph with five predefined query types:

- **patient_history:** Retrieves complete medical timeline for a patient (encounters, conditions, medications, procedures)
- **medication_info:** Returns medication details and associated conditions
- **condition_lookup:** Finds patients diagnosed with specific conditions (supports keyword search)
- **encounter_details:** Provides comprehensive encounter information
- **custom:** Allows execution of arbitrary Cypher queries for advanced use cases

The tool supports both exact code matching and partial description search, enabling flexible querying (e.g., searching for "diabetes" without knowing the exact condition code).

2.3.2 Graph Analytics Tool

Computes statistics and patterns across the knowledge graph:

- **node_counts**: Returns count of each node type in the database
- **top_conditions**: Identifies most frequently diagnosed conditions
- **top_medications**: Lists most commonly prescribed medications
- **patient_demographics**: Provides age and gender distribution
- **encounter_stats**: Analyzes encounter frequency and types

2.4 Multi-Step Reasoning

The agent demonstrates true multi-step reasoning by:

1. Analyzing complex queries that require multiple data points
2. Selecting appropriate tools based on query semantics
3. Interpreting tool results and determining if additional information is needed
4. Combining information from multiple tool invocations
5. Synthesizing coherent answers from disparate data sources

For example, when asked "How many patients have diabetes?", the agent might:

1. Use `condition_lookup` to find patients with diabetes
2. Use `graph_analytics` to get aggregate statistics
3. Synthesize both results into a comprehensive answer

The system uses Groq's Llama 3.3 70B model, which provides fast inference and strong function-calling capabilities essential for reliable tool use. The agent workflow is exposed through a FastAPI REST API, enabling integration with web applications and other services.

3 GraphRAG: Hybrid Retrieval Architecture

GraphRAG (Graph Retrieval-Augmented Generation) represents an advanced querying paradigm that combines semantic vector search with graph traversal to enable more sophisticated information retrieval from the healthcare knowledge graph.

3.1 GraphRAG Motivation

Traditional graph queries face limitations when handling complex medical queries:

- **Exact matching limitations**: Cypher queries require precise codes or keywords, failing on fuzzy or semantic queries
- **Semantic blindness**: Graph traversal alone cannot find conceptually similar but syntactically different entities
- **Isolated approaches**: Using only semantic search misses rich relational context; using only graph traversal misses semantic similarity

GraphRAG addresses these limitations by integrating both approaches into a unified retrieval pipeline.

3.2 Vector Embedding Infrastructure

The GraphRAG system is designed to utilize vector embeddings for semantic similarity search, with a flexible architecture supporting multiple embedding providers:

- **Embedded node types:** Condition, Medication, Procedure, Observation, Encounter, CarePlan
- **Embedding storage:** Vector embeddings stored as node properties in Neo4j
- **Vector indexes:** Neo4j vector indexes enable efficient similarity search using cosine similarity
- **Multi-tier fallback:** OpenAI embeddings → sentence-transformers → hash-based embeddings

Current Implementation Status: Due to storage and API cost constraints, the repository currently uses hash-based embeddings as a placeholder. These embeddings enable the infrastructure to function but **do not provide semantic understanding**. Consequently, semantic similarity search capabilities are not yet operational in the current deployment.

To enable true semantic search, users must run the `setup_graphrag.py` script with either:

- OpenAI API key for `text-embedding-3-small` (1536 dimensions, highest quality)
- Sentence-transformers library for `all-MiniLM-L6-v2` (384 dimensions, local, no API required)

The hash-based approach serves as a lightweight fallback that maintains system functionality for graph traversal operations while awaiting proper semantic embedding generation.

3.3 GraphRAG Retrieval Modes

The GraphRAG tool implements three retrieval strategies:

3.3.1 Semantic-Then-Traverse

1. **Phase 1 - Semantic Retrieval:** Query text is embedded and vector similarity search identifies top-k most similar entities (e.g., conditions similar to "diabetic complications")
2. **Phase 2 - Graph Traversal:** Starting from retrieved entities, traverse graph relationships for specified depth (1-3 hops) to discover connected nodes (e.g., medications, procedures, patient cohorts)
3. **Phase 3 - Synthesis:** Combine semantic matches with graph-discovered relationships

Best for: Queries requiring semantic understanding followed by relationship exploration ("What treatments exist for diabetes-like conditions?").

3.3.2 Traverse-Then-Semantic

1. **Phase 1 - Graph Traversal:** Execute graph pattern matching or traversal from known starting points (e.g., all medications for a condition)
2. **Phase 2 - Semantic Ranking:** Embed query and rank traversal results by semantic relevance

-
3. **Phase 3 - Filtered Results:** Return top-k semantically relevant items from graph traversal

Best for: Structural queries with semantic refinement ("Among all heart medications, find those most similar to aspirin").

3.3.3 Hybrid-Combined

1. **Parallel Execution:** Simultaneously run vector similarity search and graph traversal
2. **Result Merging:** Combine results from both approaches using weighted scoring
3. **Deduplication:** Merge overlapping entities and aggregate relationship information
4. **Relevance Ranking:** Final ranking considers both semantic similarity scores and graph centrality

Best for: Complex multi-faceted queries requiring both semantic understanding and structural exploration.

3.4 GraphRAG Implementation

The GraphRAG tool accepts the following parameters:

- **query_text:** Natural language query description
- **retrieval_mode:** Strategy selection (semantic_then_traverse, traverse_then_semantic, hybrid_combined)
- **focus_entity:** Primary entity type (condition, medication, procedure, patient)
- **traversal_depth:** Graph traversal hop count (1-3)
- **top_k:** Number of top results to retrieve
- **similarity_threshold:** Minimum cosine similarity for vector search (0-1)

The tool orchestrates the execution of both semantic search queries (using Neo4j vector indexes) and Cypher traversal queries, intelligently combining results based on the selected mode.

4 LLM Tool Utilization Workflow

The agent system provides the LLM with five specialized tools, each designed for specific query patterns. The LLM intelligently selects and chains tools based on query semantics and intermediate results.

4.1 Tool Ecosystem

Note on Current Limitations: While all five tools are fully implemented, the vector-based tools (Vector Similarity Search and GraphRAG) currently use hash-based embeddings that lack semantic understanding. To unlock their full semantic capabilities, proper embeddings must be generated via the setup script. Graph-based tools (Neo4j Query, Analytics, Graph Traversal) are fully operational.

4.1.1 1. Neo4j Query Tool

Purpose: Direct entity lookups using predefined Cypher queries or custom queries.

Status: Fully operational

Use cases:

- Patient history retrieval by exact patient ID
- Medication information lookup by code or description keyword
- Condition-based patient search with keyword matching
- Encounter details for specific encounter IDs
- Custom Cypher execution for advanced queries

Limitations: Requires exact IDs or keywords; no semantic understanding.

4.1.2 2. Vector Similarity Search Tool

Purpose: Semantic search for medically similar concepts using vector embeddings.

Status: Infrastructure complete, awaiting semantic embeddings (currently hash-based)

Use cases:

- Finding conditions semantically similar to a description ("conditions like chronic kidney disease")
- Discovering medications with similar therapeutic effects ("pain relievers similar to ibuprofen")
- Identifying similar procedures or observations

Supported node types: Condition, Medication, Procedure, Observation.

Output: Top-k similar entities with similarity scores (0-1 range).

4.1.3 3. Graph Traversal Tool

Purpose: Complex multi-hop graph navigation to discover patterns and relationships.

Status: Fully operational

Traversal types:

- **patient_journey:** Complete temporal medical timeline for a patient
- **condition_treatment_path:** Medications and procedures commonly used for a condition
- **medication_interactions:** Co-prescribed medications and common conditions
- **similar_patient_cohort:** Patients with similar medical histories
- **temporal_patterns:** Sequential medical events and progression patterns

Parameters: Traversal type, max_hops (1-5), entity identifiers.

4.1.4 4. Graph Analytics Tool

Purpose: Aggregate statistics and population-level insights.

Status: Fully operational

Analytics types:

- Node counts across all entity types
- Top conditions by diagnosis frequency
- Most prescribed medications
- Patient demographics (age, gender distributions)
- Encounter statistics and patterns

Use cases: Population health queries, trend analysis, database exploration.

4.1.5 5. GraphRAG Tool

Purpose: Hybrid retrieval combining semantic search and graph traversal.

Status: Graph traversal component operational, semantic component awaiting proper embeddings

Use cases:

- Multi-entity queries ("diabetic patients on insulin with kidney disease")
- Complex medical questions requiring both semantic and structural reasoning
- Exploratory queries with fuzzy criteria

Modes: Semantic-then-traverse, traverse-then-semantic, hybrid-combined.

4.2 LLM Decision-Making Process

The LLM follows a systematic approach to tool selection and utilization:

4.2.1 Query Analysis

Upon receiving a user query, the LLM:

1. **Parses intent:** Identifies whether query seeks specific entities, semantic similarity, relationships, or statistics
2. **Extracts entities:** Identifies mentioned conditions, medications, patient IDs, or medical concepts
3. **Determines complexity:** Classifies as simple lookup vs. complex multi-step query
4. **Assesses information needs:** Decides if single tool or multiple tools required

4.2.2 Tool Selection Strategy

The LLM applies the following decision logic:

- **Exact ID present:** Use Neo4j Query Tool with patient_history or encounter_details
- **Semantic query:** Use Vector Similarity Search for fuzzy concept matching
- **Relationship exploration:** Use Graph Traversal Tool for multi-hop navigation
- **Statistical query:** Use Graph Analytics Tool for aggregate data
- **Complex hybrid query:** Use GraphRAG Tool for integrated retrieval

4.2.3 Multi-Tool Orchestration

For complex queries, the LLM chains multiple tools sequentially:

Example workflow for "What medications are commonly prescribed for diabetes patients?":

1. **Step 1:** Vector Similarity Search → Find conditions similar to "diabetes"
2. **Step 2:** Graph Traversal (condition_treatment_path) → Discover medications for identified conditions
3. **Step 3:** Graph Analytics → Compute prescription frequencies
4. **Step 4:** Synthesis → Aggregate results and generate natural language answer

4.2.4 Iterative Refinement

The LLM demonstrates adaptive reasoning:

- **Result evaluation:** After each tool call, assesses if sufficient information obtained
- **Follow-up queries:** Issues additional tool calls if gaps remain
- **Error handling:** Adjusts strategy if tool returns empty results or errors
- **Context accumulation:** Maintains conversation history to avoid redundant queries

4.3 System Integration

The complete agent system is exposed through a FastAPI REST API with two primary endpoints:

- **POST /ask:** Accepts user query, invokes agent workflow, returns natural language answer
- **GET /graph-info:** Provides knowledge graph metadata (node counts, relationship counts, statistics)

The API enables integration with web frontends, mobile applications, or other downstream services requiring healthcare knowledge graph access.

5 LLM Prompting and Context Management

The agent system employs sophisticated prompting strategies and context management techniques to ensure reliable tool usage and coherent multi-turn interactions.

5.1 System Prompt Engineering

The LLM receives a comprehensive system prompt that defines:

- **Role definition:** "Healthcare data assistant with access to Neo4j knowledge graph"
- **Capability enumeration:** Explicit listing of all 5 tools and their purposes
- **Tool selection guidelines:** When to use each tool based on query characteristics
- **Response formatting:** Instructions for user-friendly output generation

-
- **Behavioral constraints:** Clarifying questions for ambiguous queries, accuracy requirements

The system prompt is prepended to every conversation, ensuring consistent agent behavior across interactions.

5.2 Context Window Management

The agent maintains conversation state through LangGraph’s message-based architecture:

- **Message types:** HumanMessage (user), AIMessage (agent), ToolMessage (tool results)
- **Persistent history:** All messages accumulate in the state, providing full context
- **Sequential reasoning:** Each agent invocation has access to complete conversation history
- **Tool result integration:** Tool outputs automatically appended as structured messages

This architecture prevents redundant tool calls and enables the agent to build upon previous results.

5.3 Temperature and Model Configuration

The system uses Groq’s Llama 3.3 70B model with:

- **Temperature: 0.0:** Ensures deterministic, consistent responses
- **Function calling mode:** Native tool calling support for reliable tool selection
- **Max iterations: 10:** Prevents infinite loops while allowing multi-step reasoning

Low temperature prioritizes accuracy over creativity, appropriate for healthcare data queries.

6 Test Scenarios and Evaluation

The agent system was evaluated across multiple query categories to assess tool selection, reasoning capabilities, and answer quality.

6.1 Test Query Categories

6.1.1 1. Statistical Queries

Test: "What are the top 5 most common conditions?"

Expected behavior: Use Graph Analytics Tool

Result: Correctly selected `graph_analytics` tool, returned accurate condition frequencies

6.1.2 2. Patient-Specific Queries

Test: "Show medical history for patient [ID]"

Expected behavior: Use Neo4j Query Tool with `patient_history`

Result: Correctly identified patient ID, executed appropriate Cypher query, formatted temporal results

6.1.3 3. Multi-Step Reasoning Queries

Test: "What medications are commonly prescribed for diabetes patients?"

Expected behavior:

1. Neo4j Query Tool → Find diabetes condition
2. Graph Traversal Tool → Discover medications
3. Graph Analytics Tool → Compute frequencies

Result: Successfully chained 2-3 tools, synthesized comprehensive answer

6.1.4 4. Keyword Search Queries

Test: "How many patients have hypertension?"

Expected behavior: Use Neo4j Query Tool with condition_lookup (keyword matching)

Result: Correctly used keyword search, returned patient count

6.1.5 5. Graph Exploration Queries

Test: "What are the treatment patterns for heart disease?"

Expected behavior: Use Graph Traversal Tool with condition_treatment_path

Result: Executed multi-hop traversal, discovered medication and procedure relationships

6.2 Limitations Identified

- Semantic search tools currently inactive due to hash-based embeddings
- Ambiguous queries sometimes require user clarification
- Very complex multi-entity queries may exceed iteration limit
- Graph traversal performance degrades with large result sets

7 Challenges and Tradeoffs

The development of this healthcare agent system involved several technical challenges and design decisions.

7.1 Challenge 1: Embedding Strategy

Problem: High-quality semantic search requires proper vector embeddings, but OpenAI API costs and storage constraints limited deployment options.

Solution: Implemented 3-tier fallback architecture:

- OpenAI embeddings (best quality, requires API key)
- Sentence-transformers (good quality, local, free)
- Hash-based embeddings (infrastructure placeholder, no semantics)

Tradeoff: Current deployment uses hash-based embeddings. Full functionality requires one-time setup with proper embeddings.

7.2 Challenge 2: Graph Schema Design

Problem: Healthcare data has complex many-to-many relationships that could lead to redundant edges and data duplication.

Solution: Adopted encounter-centric model where all clinical events connect through encounters rather than directly to patients.

Tradeoff: Simplified schema and eliminated redundancy, but requires one extra hop for patient-to-event queries.

7.3 Challenge 3: Tool Granularity

Problem: Deciding between few general-purpose tools vs. many specialized tools.

Solution: Implemented 5 tools with distinct purposes:

- 3 fully operational (Neo4j Query, Analytics, Graph Traversal)
- 2 awaiting semantic embeddings (Vector Search, GraphRAG)

Tradeoff: More tools increase LLM decision complexity but provide finer-grained control and better separation of concerns. The agent handles tool selection reliably with clear descriptions.

7.4 Challenge 4: LLM Selection

Problem: Balancing cost, speed, and function-calling reliability.

Solution: Selected Groq's Llama 3.3 70B:

- Free tier with generous limits
- Fast inference (tokens/second)
- Strong function-calling support

7.5 Challenge 5: Data Sampling

Problem: Full SyntheticMass dataset is massive (millions of records), impractical for development and demonstration.

Solution: Extracted representative sample covering all entity types while maintaining relationship integrity.

Tradeoff: Smaller dataset limits statistical analysis depth but enables faster queries and manageable storage. Sample is sufficient for demonstrating system capabilities.

7.6 Future Improvements

- **Caching:** Implement query result caching for frequently asked questions
- **Streaming:** Add streaming responses for long-running queries
- **Authentication:** Add API key authentication for production deployment
- **Monitoring:** Integrate observability tools for agent behavior tracking
- **Fine-tuning:** Custom fine-tune LLM on medical terminology for better tool selection

8 Conclusion

This project successfully implements a comprehensive healthcare knowledge graph querying system combining:

- **Neo4j graph database:** 9 node types, 10 relationship types, encounter-centric architecture
- **LangGraph agent:** ReAct paradigm with multi-step reasoning capabilities
- **5 custom tools:** Neo4j Query, Analytics, Graph Traversal, Vector Search, GraphRAG
- **FastAPI REST API:** Production-ready endpoints for integration

The modular architecture enables easy extension with additional tools, alternative LLM providers, or enhanced retrieval strategies.